

Scaling Trust in the Age of AI

✨ Global Hackathon Challenges & Guidelines

Version: 1.0

Last updated: 5 Nov 2025

TABLE OF CONTENTS

Challenge 1: Grokipedia Vs Wikipedia – Content Comparison and Trust Annotation

-  Challenge summary
-  Objectives
-  Technical knowledge & skills
-  Suggested tools and resources
-  Expected impact

Challenge 2: Decentralized Community Notes – Tackling Misinformation and Deepfakes

-  Challenge summary
-  Objectives
-  Suggested tools & resources

Challenge 3: Social Graph Reputation

-  Challenge summary
-  Objectives
-  Suggested tools & resources

Challenge 4: Wild card

-  Challenge summary
-  Suggested tools & resources

Participation guidelines

-  Team composition
-  Use of existing projects
-  Timeline

4.  Support

✓ **Eligibility and submission requirements**

 Three-layer architecture

 Submission requirements

 Judging framework

 Core design principles

 Judging criteria

 Terms & Conditions

Challenge 1: Grokipedia Vs Wikipedia

— Content Comparison and Trust Annotation

Challenge summary

In an age when AI-generated knowledge bases emerge to rival human-curated sources, how do we ensure that truth and objectivity prevail? This challenge invites Web3 developers, AI researchers, and students to critically compare **Grokipedia** – xAI's new AI-generated encyclopedia – against **Wikipedia**, the longstanding crowdsourced encyclopedia.

Participants will:

- **Query a fixed set of topics** (published as Knowledge Assets on the OriginTrail Decentralized Knowledge Graph by Umanitek) from both Grokipedia and Wikipedia.
- Then **analyze the content for alignment, accuracy, and completeness**. The goal is to uncover discrepancies (if any) in Grokipedia's AI-written entries – such as factual errors, missing context, hallucinations, or biases – and to verify where the two sources agree.

- Ultimately, participants will **record their findings as Community Notes** on the decentralized knowledge base, creating a trust layer of verifiable context that AI systems and humans alike can rely on.

Example of content parity:

- The PlayStation 5 article on Wikipedia versus Grokipedia shows that Grokipedia often replicates Wikipedia content under a Creative Commons license.
- Grokipedia is an AI-generated encyclopedia launched by Elon Musk's xAI as an alternative to Wikipedia. In fact, Grokipedia already hosts over **885,000 AI-written entries**, versus Wikipedia's **~7 million English articles**. Many Grokipedia pages are adapted nearly verbatim from their Wikipedia counterparts, and entries even claim to be "fact-checked" by Grok (xAI's LLM) despite large language models' tendency to fabricate facts.

However, on *controversial topics*, Grokipedia's content can *diverge notably* – often inserting conservative-leaning viewpoints or unfounded claims not found on Wikipedia:

- For example, Wikipedia's article on climate change emphasizes the scientific consensus on human-driven warming.
- Whereas Grokipedia's version plays up critics who dispute the "near-unanimous" consensus and accuses the media and advocates of exaggeration.

These differences underscore the need for **truth alignment**. By systematically comparing the two knowledge sources, we can highlight where the AI might be hallucinating or be biased and provide corrective context.

Objectives

Participants in "**Grokipedia Vs Wikipedia: Content Comparison and Truth Alignment**" will tackle the following objectives:

1. Data retrieval

Query and retrieve **content for the designated set of topics** from both Grokipedia and Wikipedia. This includes collecting **textual content and any available media** (images or videos) associated with each topic on both platforms. Ensure you use the provided Knowledge Asset identifiers on the OriginTrail DKG to locate the topics, and/or public APIs where applicable (e.g., Wikipedia's or Wikidata's APIs).

2. Content similarity analysis

Measure and **compare the content across modalities** – text, images, and video. Determine how similar or different each Grokipedia entry is to its Wikipedia counterpart. This may involve natural language processing to compute text similarity, checking if images or illustrations are the same or different, and comparing any video or audio references. The aim is to quantify content overlap and spot where Grokipedia diverges from Wikipedia.

3. Discrepancy identification

Identify and document any **discrepancies** uncovered. Focus on:

- Factual inconsistencies (e.g., Grokipedia stating a claim that contradicts Wikipedia or other known sources),
- Missing context or sections,
- Signs of AI hallucinations (fabricated “facts” unsupported by evidence),
- Signs of bias in tone or framing.

For instance, does Grokipedia omit important caveats that Wikipedia includes? Does it introduce claims that lack citation or reflect a partisan slant? Flag these differences clearly.

4. Community Notes creation (structured)

For each discrepancy **or** each topic reviewed, **create a Community Note that summarizes** your findings. Each note should be created as Knowledge Assets on the DKG so that it can be ingested into knowledge graphs. The note should capture:

- The topic,
- A description of the discrepancy or alignment,
- References or evidence (e.g., citing Wikipedia passages or external sources for verification).

If a topic shows no major discrepancies and the content is aligned, the **note can confirm that alignment** (this is just as important for establishing trust). The structure might follow schemas for fact-checking or Web annotations, but you are free to design a lightweight schema as long as it’s clear and linkable.

5. Publishing to the DKG with trust metrics

Publish the Community Notes to the OriginTrail Decentralized Knowledge Graph (DKG) so that they become part of the **tamper-proof, verifiable knowledge network**. This will likely involve converting your JSON-LD or RDF data into a Knowledge Asset and using OriginTrail's publishing protocol.

Additionally, you are encouraged to **integrate trust metrics** into the notes using **tokenomics**. For example, you might simulate upvoting or credibility scoring of notes by linking a **NEURO** or **TRAC** token stake to each note (e.g., higher token weight = higher confidence). This optional step explores how decentralized incentives can enhance data quality – think of it as community curation where contributors “stake” tokens to vouch for a note’s accuracy or importance.

6. MCP integration for AI agents

Leverage the **Model Context Protocol (MCP)** to **demonstrate how AI agents** can utilize the annotated knowledge base for improved alignment.

In practice, this means showing that an AI (such as an LLM-based assistant or agent) can:

- **Query the DKG for relevant** Community Notes when asked about one of the topics, and
- **Adjust its response** based on that trusted context.

For example, if a user asks the AI about a topic where Grokipedia had a mistake, the agent (via MCP) should fetch the community note highlighting the correction from the DKG and incorporate that into its answer.

This objective will showcase dynamic interaction between AI and the Web3 knowledge layer – a concrete step toward AI alignment with verified truth.

7. x402 monetization

For an extra challenge, implement the **x402 payment protocol** to monetize access to the verified high-confidence notes. This could involve setting up an HTTP 402 (“Payment Required”) flow or a similar mechanism in which an **AI agent or user must pay a tiny fee** (e.g., a stablecoin microtransaction) to retrieve premium

verified information. Participants might simulate this with test tokens or a sandbox environment.

The purpose of this optional task is to explore novel web monetization models for Knowledge Assets – imagine an ecosystem where contributing high-quality truth-checks is rewarded, and consumers (including AI services) pay micropayments for the assurance of trusted information.

Technical knowledge & skills

- **Web3 and DKG familiarity**

Participants should have a basic understanding of decentralized networks and knowledge graphs. Prior experience with the OriginTrail DKG (or similar semantic blockchain systems) will be helpful. You'll be working with **Knowledge Assets** and possibly **interacting with blockchain transactions** (for publishing data and handling token stakes), so familiarity with concepts like DIDs, content addressing, and smart contracts is a plus.

- **Data handling & APIs**

Ability to work with **web APIs** or **scraping tools** to fetch content from Wikipedia and Grokipedia. Wikipedia offers well-documented REST and GraphQL APIs for retrieving page content, and so does its knowledge graph counterpart Wikidata (which is already structured in RDF). Grokipedia may not have an official API; you might need to scrape HTML or use any provided data dumps/Knowledge Assets. Comfort with parsing HTML/JSON and handling large text data will be required.

- **Natural Language Processing**

Some experience with NLP or text analysis is expected. You might use **embedding models** or **similarity algorithms to compare texts**. Knowledge of **libraries** (Python's spaCy, Transformers, or JS NLP libraries) or **AI services** (e.g., OpenAI embeddings, Cohere, etc.) for computing semantic similarity will be beneficial. Likewise, understanding how to **compare** or **hash images** (for example, using perceptual hashing or CLIP embeddings) and how to extract key frames or metadata from videos could be useful for the multi-modal comparison aspect.

- **Structured data (JSON-LD/RDF)**

Familiarity with linked data formats is important. You should be comfortable

constructing data in **JSON-LD** or basic **RDF triples** (e.g., subject-predicate-object statements). Know how to use or import common vocabularies if needed (schema.org, Web Annotation, etc.), or at least create a consistent JSON structure that can represent the Community Notes. Knowledge of tools like RDFLib, Apache Jena, or even simple JSON construction will come in handy.

- **Programming and scripting**

General programming skills are a must (JavaScript, Python, or similar). You will be writing scripts or applications to perform comparisons, format output, and interact with the DKG. If you plan to integrate MCP and x402, you'll likely be writing some backend logic or smart contract calls. Ensure you can run a local environment for testing (Node.js or Python environment, plus any required SDKs for the DKG).

- **AI integration (MCP)**

Understanding **how to interface an AI model with external context** (via the Model Context Protocol or analogous methods) is expected for the MCP part. This could involve using an AI agent framework or API (for example, OpenAI's function calling, Anthropic's Claude with tools, LangChain, etc.). You should be able to orchestrate a prompt or an agent query so that it calls out to your knowledge graph (via SPARQL query, GraphQL, or a search API on the DKG) and then processes the returned information. Familiarity with JSON APIs and parsing will be needed here.

- **Blockchain/tokens**

If attempting the tokenomics and payment integration aspects, you'll need a basic grasp of how to **simulate or implement token transfers or micropayments**. This might involve interacting with a **Polkadot-based blockchain – NeuroWeb** and using wallets or SDKs to simulate staking or payments. No deep smart contract development is necessary (unless you choose to extend functionality), but understanding how to call a payment API or sign a transaction will be useful.

Note: The hackathon organizers will provide documentation for the OriginTrail DKG and Umanitek's Knowledge Assets, including the list of topics to focus on. Make sure to review those resources and set up the required API keys or network endpoints early.



Suggested tools and resources

- **DKG Edge Node and Knowledge Assets (hard requirement)**

Leverage the DKG by building your project using the DKG Edge Node, which provides tools for Knowledge Asset creation and querying, agentic features based on MCP, which you can leverage, and a generic UI. Build your custom code on top. Use Knowledge Assets as your knowledge/data containers, reference them by UALs (Uniform Asset Locators).

This is a hard requirement for all submissions. Learn more about the DKG Edge Node and Knowledge Assets in [the official documentation](#) (updated Nov 5!).

- **Grokipedia Website**

The [Grokipedia](#) site will serve as a primary source for AI-generated content. Use it to retrieve topic pages. (💡 Tip: If no API is available, consider using web scraping tools or see if Umanitek's DKG assets include Grokipedia extracts.) Keep an eye on the site for any indicators (like "Fact-checked by Grok" timestamps) that might inform your analysis.

- **Wikipedia API and dumps**

Leverage Wikipedia's official API ([MediaWiki API](#)) to fetch page content in a structured format (JSON or XML). This will make it easier to parse the text and media info for each topic. Alternatively, for faster access, consider using Wikipedia data dumps or tools like the Wikipedia Python library. Wikipedia's content will serve as the baseline "ground truth" for comparison.

- **OriginTrail DKG documentation**

Refer to [OriginTrail's developer docs](#) on how to publish and query Knowledge Assets. This includes understanding how to format data in JSON-LD, how to use the DKG client or GraphQL interface, and how to ensure your data is discoverable. The concept of **Knowledge Assets** (self-contained packets of verified data) will be central. The docs also cover how provenance and verifiability are handled, which is useful for the trust aspect.

- **JSON-LD and RDF tools**

If you're new to JSON-LD, check out the W3C JSON-LD primer or [schema.org](#)

documentation for examples. Tools like [JSON-LD Playground](#) can help validate your JSON-LD structure.

- **NLP and similarity libraries**

To compare texts, consider using embedding models. However, more interesting comparisons would occur on structured statements in Knowledge Assets from both sources, effectively comparing triples (statements) rather than purely textual similarity comparisons. A hybrid approach, such as comparing knowledge graph embeddings, would be even more interesting. To detect hallucinations or unsourced claims, you might cross-reference each statement in Grokipedia with the Wikipedia text or external sources (this can be semi-automated by searching for sentences). You can also leverage the symbolic reasoning features of the DKG Edge Node. For comparing images, tools like **ImageHash** or **OpenCV** can detect if two images are identical or very similar (useful if Grokipedia copied images from Wikipedia or omitted them). For video content, if any, you might use video metadata or transcripts (e.g., [YouTube Data API](#) if Wikipedia includes a video reference).

- **Model Context Protocol (MCP)**

The DKG Edge Node runs an MCP server that exposes the DKG to LLM workflows. Resources like [Anthropic's documentation on MCP](#) or [OpenAI function-calling tutorials](#) could guide you. Essentially, you'll want to set up a pipeline where an AI model, given a user query, triggers a knowledge graph search in your Community Notes dataset. You can use a simple keyword search over your notes (maybe index them with ElasticSearch or even a local vector index if using embeddings) to find relevant notes, then feed that into the model's prompt. Libraries like **LangChain** or **LlamaIndex** can streamline this process by allowing you to treat the DKG as a knowledge retriever.

- **x402 payment protocol info**

If attempting the monetization aspect, check out resources on the [x402 protocol](#) (an HTTP-based micropayment standard). [Coinbase's blog post](#) or [PayPal's developer forums](#) may have conceptual guides on implementing HTTP 402 flows. In practice, you might use a dummy stablecoin on a testnet and set up a simple API

gateway that requires a token transfer or a signed payment message before returning the data..

- **Community Notes inspiration**

To understand how concise fact-check notes are written, refer to [X's \(Twitter's\) Community Notes program](#). While those are not structured data, they can give insight into tone and clarity – each note typically provides context and evidence without ranting. Aim for a neutral, clear summarization of discrepancies in your notes, much like a Wikipedia editor or fact-checker would.

- **Collaboration and support**

Don't forget the hackathon's [Discord channel](#), where mentors might share additional resources or answer questions about the DKG, tokens, or Grokipedia quirks. Utilize version control (GitHub) for your project, especially since you might be dealing with data and code – it will help in sharing your solution during judging.

Expected impact

By participating in this challenge, you will contribute to **building a “safety net” for AI-generated knowledge** – an emerging necessity as AI systems like Grok are increasingly used to produce information. The work done here has several far-reaching impacts:

1. **AI alignment with human knowledge**

This project operationalizes AI alignment by linking an AI's outputs (Grokipedia content) with an alternate knowledge source (Wikipedia) and flagging misalignments. Demonstrating the MCP-driven feedback loop shows a path to AI that can self-correct using decentralized knowledge resources, reducing the spread of AI hallucinations and bias.

2. **Empowering decentralized fact-checking**

Just as Wikipedia revolutionized encyclopedic knowledge through global collaboration, this challenge extends that spirit to *fact-checking* in the AI era. The creation of Community Notes in a **decentralized knowledge graph** means no single authority is policing truth; instead, a transparent, community-driven process does, with verifiability baked in. Over time, such a system could scale to

address deepfakes, fake news, and any AI-generated misinformation, as the notes can capture not only textual discrepancies but also contextual information about images or videos. DKG Paraneets would enable the incentivization of such contributions at scale.

3. Trust layer for the Internet

Your work aligns with Umanitek's vision of a "*trust layer*" for AI – embedding provenance, transparency, and accountability into data that AI systems use. By publishing notes with provenance (e.g., citing sources and anchoring them on the blockchain), you make it possible to trace the origin of information and its verification status. This is a concrete step towards a more **trusted Web**, where content is accompanied by machine-readable "nutrition labels" or fact-check annotations.

4. Innovating knowledge economics

Exploring tokenomics and payment for knowledge (via NEURO, TRAC, x402) could herald a new model for sustaining public knowledge. Imagine a future Wikipedia where contributors are rewarded for accuracy, or a knowledge graph where users pay tiny fees for high-confidence answers (and those fees flow back to the validators of information). By prototyping this, you're testing how to **incentivize truth** in a way that both motivates contributors and deters spam (since staking tokens can carry a cost). This could make the ecosystem self-sustaining and resistant to manipulation.

5. Enhancing Grokipedia and Wikipedia

The findings from this hackathon could inform both the Grokipedia and Wikipedia communities constructively. For *Grokipedia* (and xAI), the identified discrepancies are valuable feedback to improve their AI model or adjust their content generation to avoid mistakes. For *Wikipedia*, seeing where an AI diverges highlights the strengths of human oversight and may pinpoint areas to double down on (such as reinforcing certain facts or making articles more robust against misinterpretation). In a sense, this competition may lead to *collaboration*: an AI-generated encyclopedia and a human-curated one can evolve together, with our DKG-based truth alignment layer facilitating communication between them.

Challenge 2: Decentralized Community Notes — Tackling Misinformation and Deepfakes

Challenge summary

Build a solution that uses the **OriginTrail Decentralized Knowledge Graph (DKG)** and **Umanitek Guardian** technology to create a **community-driven fact-checking system** (akin to “Community Notes”) on a decentralized network.

The goal is to **combat fake news, deepfakes, and misinformation** by combining:

- Trust layer
- Knowledge graphs
- AI agents
- x402 payment protocol

Your challenge is to enrich a provided **Social Graph** knowledge base of combined Twitter, Reddit, YouTube, and TikTok content on the DKG and demonstrate how token incentives and decentralized tools can **enhance truth verification** in the AI era.

Objectives

Teams should address **4 key components** in their projects:

1. Implement a tokenomics-powered trust layer

Incorporate a token or incentive system to establish trust and reputation for community contributions. Use Web3 **tokenomics** (e.g., OriginTrail's \$TRAC, \$DOT, \$NEURO, or custom tokens) to reward users/agents for adding accurate notes or verifying content.

This “*trust layer*” will serve as an **infrastructure that protects truth** and **safeguards reputations** in the face of deepfakes. For example, contributors might stake tokens when submitting a note (to signal confidence) and earn rewards if others validate their note.

This economic layer should enrich the Community Notes knowledge base by incentivizing high-quality contributions and deterring misinformation.

2. Enrich the knowledge base with knowledge graph triples

Extend the provided **Umanitek Guardian sample Social Graph** (a DKG Knowledge Asset repository) with new facts, context, and relationships in the form of knowledge graph triples.

In practice, this means adding Community Notes as **RDF triples** (subject–predicate–object statements) in the **DKG’s knowledge layer**. Each note or fact-check should be represented semantically (as JSON-LD) and linked to the content it annotates. The OriginTrail **DKG Edge Node** can be used to transform various data (documents, web content, etc.) into “semantically rich and connected Knowledge Assets”; use it to publish your new triples. By structuring notes as linked data, they become part of a **global, interoperable graph** that AI agents and users can query.

This enrichment should upgrade the existing Guardian Knowledge Assets with fresh data (e.g., adding new verified claims, linking a deepfake video to its fact-check note, etc.), thereby continuously improving the collective knowledge on the DKG.

3. Leverage AI agents and MCP for querying & knowledge generation

Demonstrate an AI agent that interacts with the DKG to both **use and augment the Community Notes** knowledge base. Use the **Model Context Protocol (MCP)** toolkit (or similar agent integration tools) to connect your agent to the DKG and the Umanitek Guardian sample Social Graph. MCP is essentially a “USB-C for AI” – a standard that lets AI models (like LLMs) plug into external tools and data sources.

By integrating the DKG as an MCP tool, your agent can **query the knowledge graph in real time** (e.g., use SPARQL queries to retrieve notes or verify if a claim was flagged) and **generate new knowledge** to publish back to the graph. For

instance, the agent might detect a suspicious image or statement, use the Guardian Social Graph to check for known deepfakes, then create a new Community Note triple if needed.

Your solution should **showcase an agent-driven workflow**:

- The agent queries the DKG for relevant info,
- Reasons about misinformation, and then
- Adds a verified note or alert as a new Knowledge Asset.

This creates a **knowledge-graph-based, collective, persistent memory** for agents on the DKG. Essentially, each team's agent becomes a Guardian that both consumes and contributes to the decentralized truth repository.

You are encouraged to use or adapt the provided Umanitek Guardian Agent sample Social Graph and MCP integration examples (e.g., the dkg-mcp-server template) to jumpstart development.

4. Integrate the x402 protocol for data transactions

Incorporate the x402 standard to enable **transactions or micropayments for accessing high-quality data** (premium knowledge assets) within your solution.

The x402 protocol is an open standard that revives the HTTP 402 "Payment Required" code to enable clients (humans or AI agents) and servers to **negotiate micropayments in real time**. In the context of this challenge, x402 can be used to purchase or pay for access to trusted data sources that enhance your Community Notes knowledge base.

For example, if an agent needs a verified news article or a deepfake detection API result that isn't freely available, using x402 would allow it to automatically pay (using stablecoins or tokens) and retrieve that information. Teams should propose how x402 transactions **add "quality context"** to the knowledge base – perhaps by gating certain high-confidence data behind a micro-paywall accessible only to well-intentioned agents (with tokens), thereby adding an extra layer of assurance. You might simulate an AI agent hitting a 402 Payment Required response when querying a premium data API, then using an x402 payment to fetch the data and feed it into the DKG.

The aim is to demonstrate **synergy between decentralized knowledge and decentralized payments**: enabling a sustainable model for truth-seeking agents to get verifiable, high-quality information on demand. This also introduces an economic layer where data providers can be rewarded for supplying reliable knowledge to the network.

Suggested tools & resources

To build your solution, consider leveraging the following resources:

- **OriginTrail DKG Edge Node (hard requirement)**

Learn how to run an Edge Node and publish/query knowledge assets on the DKG. The Edge Node allows you to transform data into verifiable, linked Knowledge Assets and provides APIs for searching and updating the knowledge graph. Using the DKG Edge Node is a hard requirement for all submissions. Learn more about the DKG Edge Node and Knowledge Assets in the [official documentation](#) (updated Nov 5!).

- **Umanitek Guardian knowledge base & access to Guardian AI Agent**

A sample knowledge base of “Guardian” Knowledge Assets will be provided, containing known fingerprints of illegal or deepfaked content (from Umanitek’s demo). Use this as a starting point – your Community Notes should build on it and extend it. Umanitek’s Guardian AI Agent showcases how an agent can identify deepfakes using the DKG, which you can replicate or build upon. Refer to any provided dataset or API from Umanitek for detecting fake media and illegal content.

- **MCP toolkit for agent-DKG integration**

Documentation and examples for using the **Model Context Protocol (MCP)** with the DKG are available (e.g., [OriginTrail's dkg-mcp-server example on GitHub](#)). MCP enables your AI agent to query and update the DKG through standardized tool interfaces. This toolkit will help **connect your LLM or agent framework** (such as LangChain, GPT-4, Claude, etc.) to the DKG so it can read/write Community Notes. See [OriginTrail's documentation](#) for how an agent can execute SPARQL queries and publish JSON-LD assertions via MCP.

- **x402 protocol guides**

The [x402 Foundation's site](#) and recent tutorials contain details on implementing the payment protocol. Review how to handle a *402 Payment Required* response and complete a micropayment transaction (using a crypto wallet or stablecoin API) to retrieve a resource. You might use sandbox tools or dummy stablecoins for demonstration. A helpful reference is the [x402 Telegram Bot](#) example, which uses x402 for pay-per-use content access – this can inspire how your agent will pay for data.

- **Additional resources**

Additional resources include the [OriginTrail Whitepaper](#) (for understanding DKG's design and tokenomics), [schema.org/RDF](#) references (to model your triples), and any **AI libraries** for building the agent. Teams are **free to bring their own datasets** or **AI models** to complement the provided tools – creativity is encouraged!

- **Collaboration and support**

Don't forget the hackathon's [Discord channel](#), where mentors might share additional resources or answer questions about the DKG, tokens, or Grokipedia quirks. Utilize version control (GitHub) for your project, especially since you might be dealing with data and code – it will help in sharing your solution during judging.

Challenge 3: Social Graph Reputation

Challenge summary

Online platforms today grapple with **bot accounts**, **misinformation**, and **a lack of trust** in user identities. Decentralized technologies offer new ways to tackle these challenges by combining **tokenomics**, **social graph analysis**, and **web3 microtransactions**.

For example, decentralized reputation systems can reward genuine contributions and limit fake identities, while **proof-of-personhood** techniques use cryptographic tokens to

ensure each user is a unique human. In parallel, graph algorithms like PageRank have long been used to measure reputation or influence in networks (e.g., ranking websites, academic papers, or social profiles). However, graph-based reputation alone can be gamed by Sybil attacks (e.g., a single entity making many fake nodes that all upvote each other) – motivating the need for **token-economic signals** to distinguish real users from impostors.

This hackathon challenge, “**Social Graph Reputation**,” invites you to build a solution that combines blockchain **tokenomics with social network graph analysis** to establish **user reputation**, and then leverages the new **x402** payment standard to enable trusted transactions based on that reputation.

The goal is to infer which accounts are **trustworthy or “real” versus fake**, using both token-based verification and graph-based reputation scoring, and demonstrate how this reputation can facilitate **quality data exchange** or **e-commerce transactions** in a decentralized way. By integrating the **OriginTrail Decentralized Knowledge Graph (DKG)** and modern Web3 tools, your solution can provide a verifiable, transparent **trust layer for social interactions** – much like Umanitek’s Guardian AI Agent, which is built on the OriginTrail DKG to verify content across distributed sources without exposing sensitive data.

Objectives

Teams are tasked with developing a prototype that addresses **1 or more of the following aspects**:

1. Tokenomics for identity & trust

Use a **token-based mechanism to signal** which accounts are **likely genuine**. This could involve requiring a **stake or credential** for each account to prove humanness, using community staking/voting to validate users, or issuing non-transferable **identity tokens** to verified humans. The idea is to raise the economic cost of creating fake accounts and provide a **quantifiable trust signal** (e.g., an account that has staked tokens or holds a “proof of personhood” NFT is more credible). Leverage the DKG if possible to store or verify these credentials (for example, a Knowledge Asset that marks an account as verified). Key considerations include Sybil attack resistance – tying identity to scarce resources (such as tokens or unique credentials) makes Sybil attacks costly. You might also

draw inspiration from **token-curated registries**, where token holders collectively maintain a list of trustworthy accounts by staking tokens on entries.

Some ideas:

- Require each user to **stake a small amount of a token** (e.g., a utility token like TRAC, DOT, or NEURO) upon account creation. Honest users won't mind a minor stake (which could be returned or earn rewards), but spammers operating many accounts would incur high costs.
- Issue a **soulbound token or NFT** to each verified user (after a specific verification process) and store it on the DKG. For example, Proof-of-Humanity-style verification could grant each human a unique token.
- Implement a **community-vetting system**: Existing trusted users spend tokens to vouch for new accounts. If an account they endorsed turns out to be fraudulent, the stakers lose their deposit – aligning economic incentives towards honest endorsements.
- Use **on-chain behavior** as a signal: E.g., if an account has transacted with many others (receiving diverse token payments or endorsements), it's more likely real. An account with zero external interactions or with all activity confined to a single cluster might be a Sybil identity.

2. Graph-based reputation inference

Utilizing the **Social Graph** structure, calculate an **algorithmic reputation score** for each account. You will be provided with a real-world social network dataset (as a knowledge graph) via the Umanitek Guardian Agent sample knowledge base on the DKG. This data may include relationships such as followers, friends, or content interactions between accounts. Use this graph to infer an “interesting signal” about reputation – for example, applying the classic PageRank algorithm (or a variant) to compute a global reputation score based on network connections. PageRank has been used effectively to assign reputation in social networks, treating an endorsement or follow as a “vote” of confidence similar to how a link between web pages signals importance. In a social graph, an account is

considered reputable if trusted accounts point to it (e.g., many credible users follow or interact with that account).

Ideas:

- Model the social network as a graph where nodes are user accounts and edges represent trust or interactions (e.g., an edge from A to B might denote “A endorses or follows B”). Then apply a centrality algorithm, such as **PageRank**, to this directed graph. The output will be a score for each node – higher if it’s pointed to by other high-scoring nodes, reflecting an iterative notion of influence or trust.
- **Combine token signals with PageRank:** You can weight the graph edges or nodes using the results from the above. For instance, give more weight to links coming from verified/staked accounts, so that bot accounts (with no token stake) have minimal influence on reputation propagation. This creates a synergy in which tokenomics provides a base layer of trust, and the graph algorithm amplifies the trust through network effects.
- Explore other graph metrics too – e.g., **degree centrality** (number of unique connections) or **community detection** to find tightly knit clusters. A cluster of accounts that only interact with each other, not with verified humans, could indicate a Sybil cluster. You could flag such anomalies as low-reputation despite any internal PageRank they exchange.
- Use the **OriginTrail DKG** to your advantage: Since the dataset is provided as a knowledge graph, you can query it with semantic queries (SPARQL) or export it for analysis. The DKG ensures data provenance and verifiability, so any reputation scores or trust labels you compute can be published back as *verifiable Knowledge Assets* on the DKG for transparency. For example, after computing reputations, you could create a Knowledge Asset that attaches a reputation score to each account’s identifier, enabling others to query “What is Alice’s reputation?” from the DKG.

3. Transactions via x402 for quality data & commerce

Finally, demonstrate how the reputation system enables **trusted transactions** in a data marketplace or e-commerce scenario by incorporating the **x402 protocol**.

The x402 standard is an **HTTP 402-based payment protocol** that allows seamless micropayments for web resources and APIs. It effectively lets servers require payment (in cryptocurrency, e.g., stablecoins) for an API call or data request, with clients (which could be users or AI agents) automatically handling the payment and retrying the request. This opens the door to **machine-to-machine commerce** and **pay-per-use data access** with no need for manual sign-ups or third-party gateways. In the context of this challenge, x402 can be used to **monetize high-quality data or services** emerging from your social graph reputation system: for instance, charging a small fee to access a “trusted content feed” or to query the top-N reputable users in a category. Because your system can quantify trust, consumers may be willing to pay for data that has a verifiable quality/reputation score. Similarly, reputable users could **sell data or services** (e.g., an influencer selling a recommendation, or a verified analyst selling a report) directly to others, with x402 handling the payments in a decentralized manner.

Implementation ideas:

- **Pay-per-API:** Develop a simple API/MCP tool that returns some premium data – for example, an endpoint that returns a **list of users with the highest reputation scores**, or detailed profile data from the DKG about a particular account’s trust metrics. Protect this endpoint with x402, such that a client must pay a tiny fee (e.g. \$0.01 in a stablecoin) to retrieve the data. You can use the official x402 libraries or middleware – as simple as adding a line like `paymentMiddleware("0xYourAddress", {"/your-endpoint": "$0.01"});` to your DKG Edge Node to require a payment. If the request comes without payment, the server responds with HTTP 402 Payment Required, and the client/agent can then automatically submit the payment and repeat the request.
- **Data marketplace scenario:** Imagine your DKG-backed social network as a marketplace where **high-reputation users provide valuable data or content**. Use x402 to let those users charge for access to their content. For instance, a trusted data provider could offer an API endpoint (or file) with curated data, and other agents pay per access. Because x402 is blockchain-agnostic and has no platform fees, all payments go directly to the provider’s wallet with minimal friction (near-instant settlement in ~2

seconds). You could simulate this by having your application serve a file (e.g., a JSON of some analysis results) that requires payment via x402.

- **E-commerce use case:** Extend the idea beyond data into e-commerce: e.g., a reputable seller in the network selling a digital good. Demonstrate an **automated purchase** where an AI agent (or user script) finds a product from a seller with a high trust score and uses x402 to buy it on the fly. x402 is designed for agent-to-agent payments, so an AI bot could negotiate a price and pay for a resource autonomously. For example, you might show a bot purchasing a “report” or unlocking an insight from the DKG after verifying the source’s reputation and paying via x402.
- **Quality data via microtransactions:** Because x402 enables micropayments without subscriptions or ads, consider how your reputation system could **filter out low-quality data and only monetize the good stuff**. Perhaps your app offers a “verified info” service where any fact or piece of content is backed by a high-rep source (with provenance on the DKG), and clients pay per query to access verified information. This would show how **trusted data becomes a commodity** – users/agents pay small amounts for each trustworthy answer, which incentivizes participants to contribute high-quality data (since they could earn from it). A real-world analogue is a Telegram bot that charges for premium content via x402; in fact, an example integration has already been implemented with a bot accepting stablecoin payments for content and API calls. You can take inspiration from that for design and implementation.

Suggested tools & resources

To help you build this, we provide and recommend the following resources:

- **OriginTrail DKG & Edge Node (hard requirement)**
OriginTrail Decentralized Knowledge Graph (DKG) is a decentralized network for organizing data into a **global, verifiable, interoperable knowledge graph**. The DKG Edge Node allows you to **ingest, query, and publish Knowledge Assets** on this graph easily via REST APIs or a web UI and launch DKG-based AI agents. You can use a DKG Edge Node to access the provided social network dataset (the Guardian Agent knowledge base) and then query relationships or add new data.

The DKG ensures any data or reputation scores you publish are cryptographically signed and timestamped (anchored on blockchains), providing an audit trail of trust.

Using the DKGEde Node is a hard requirement for all submissions. Learn more about the DKG Edge Node and Knowledge Assets in the [official documentation](#) (updated Nov 5!), including how to transform JSON data into JSON-LD (linked data format) and how to write SPARQL queries to explore the graph.

- **Umanitek Guardian Agent sample knowledge base through Guardian Agent MCP**

A **real-world social network dataset** will be provided, derived from Umanitek's Guardian Agent knowledge base on the DKG, containing Twitter, Reddit, TikTok, and YouTube content. This data can include entities such as accounts, content items, and relationships (e.g., who follows whom, content postings, or trust assertions between users). It is your primary dataset for analysis. The knowledge base is formatted in a DKG-compatible way (likely JSON-LD or RDF triples), which you can load directly into an Edge Node or query via the DKG API. This gives you an authentic graph to run reputation algorithms on, rather than having to fabricate social data.

For context: Umanitek's Guardian Agent is an AI that uses the DKG to identify and track harmful content across networks; the provided data might reflect parts of that process (for example, a network of content-sharing or user-flagging relationships that you can treat as a social graph). Use this rich dataset to test your tokenomic and graph algorithms – e.g., find clusters of accounts, run PageRank to identify the most influential nodes, etc. Since the data comes from a real use case, the insights you derive (like detecting a likely bot cluster or a highly trusted user) will be more meaningful.

- **x402 protocol documentation & tutorials**

The [x402](#) standard is new (emerging in 2025) and aims to be the “**payments layer**” **for the web**, letting any client-server pair perform **automatic payments over HTTP** – x402 uses under-the-hood integration with crypto wallets. The x402 protocol enables **machine-to-machine commerce** and micropayments for APIs in a way that earlier approaches (such as paywalls) could not. Explore how **autonomous agents** in your project might interact with each other (e.g., a

reputation bot that pays to get additional data) and how to keep transactions trust-minimized. Remember: x402 is not a coin or a blockchain itself – it's a layer on top of HTTP. This means you can focus on your web app while x402 handles payments in the background.

- **MCP Toolkit on the DKG Node Agent**

The Model Context Protocol (MCP) is another cutting-edge tool that can enhance your project. MCP is a **standardized way for AI agents** (especially LLM-based agents) to **plug into external tools and data reliably**. DKG nodes are centered around MCP, which means you could have an AI agent that queries the DKG for info, updates the knowledge graph, and even uses the reputation data as part of its reasoning. For instance, you could build a chatbot that, when asked about a user, will fetch that user's reputation score from the DKG (via a SPARQL query) and then decide whether to trust information from that user. This might be an optional advanced component of your solution: integrating an AI (like a Copilot, GPT, or similar) that uses your reputation system in real time. For example, an agent could automatically decide to pay via x402 for specific data only if the provider's reputation is above a threshold – a neat demonstration of an agent making trust-based economic decisions.

- **Collaboration and support**

Don't forget the hackathon's [Discord channel](#), where mentors might share additional resources or answer questions about the DKG, tokens, or Grokipedia quirks. Utilize version control (GitHub) for your project, especially since you might be dealing with data and code – it will help in sharing your solution during judging.



Challenge 4: Wild card



Challenge summary

This challenge is for creative thinkers and engineers who enjoy thinking outside the box.

Come up with your own ideas and challenges to solve. Any combination of the above challenges is fair game. As are any entirely out-of-the-box ideas, as long as you fulfill some basic requirements:

1. **Use and leverage the DKG Edge Node** in your project
2. **Leverage the 3 core layers** in your project:
 - a. Trust layer
 - b. Knowledge layer
 - c. Agent layer
3. **Utilize Knowledge Assets and demonstrate functional or conceptual synergy across layers.**
4. Bonus points if your solution demonstrates how **AI agents build on Polkadot's interoperability and shared security to empower the next generation of autonomous agents**, from multi-chain DeFi advisors to AI-driven governance assistants and verifiable data oracles.



Suggested tools & resources

Use any of the tools and resources suggested in the other challenges or choose your own. There is only one hard requirement:

- **OriginTrail DKG & Edge Node (hard requirement)**

OriginTrail Decentralized Knowledge Graph (DKG) is a decentralized network for organizing data into a **global, verifiable, interoperable knowledge graph**. The DKG Edge Node allows you to **ingest, query, and publish Knowledge Assets** on this graph easily via REST APIs or a web UI and launch DKG-based AI agents. The DKG ensures that any data you publish is cryptographically signed and timestamped (anchored on blockchains), providing an audit trail of trust.

Using the DKG Edge Node is a hard requirement for all submissions. Learn more about the DKG Edge Node and Knowledge Assets in [the official documentation](#)

(updated Nov 5!), including how to transform JSON data into JSON-LD (linked data format) and how to write SPARQL queries to explore the graph.

Participation guidelines

1. Team composition

You can work solo or in teams. Participants are free to use their own tools, libraries, and datasets beyond the provided resources – just disclose what you used.

Collaboration between developers, graph enthusiasts, and AI experts is recommended, given the interdisciplinary nature of the challenge.

2. Use of existing projects

Yes – you may build on existing open-source projects or prior hacks, as long as you clearly credit them and your work adds new functionality aligned with this challenge (we want to see **OriginTrail DKG, Umanitek, and Polkadot tech integrated** in a novel way).

3. Timeline

The hackathon runs for 19 days. We encourage an **early proof-of-concept** by mid-hack (e.g., agent able to query DKG or a basic triple added to the knowledge base) to validate your approach, then iterate with the trust and payment features.

4. Support

Mentors familiar with the DKG, knowledge graphs, and Umanitek Guardian will be available during the hackathon. Office hours will cover setting up an OriginTrail node, formatting JSON-LD for knowledge publishing, and simulating x402 payments. Don't hesitate to ask questions in the hackathon [Discord channel](#).

Eligibility and submission requirements

Projects **must use the DKG Edge Node** and **leverage the three-layer architecture**, utilizing Knowledge Assets and demonstrating functional or conceptual synergy across the Agent-Knowledge-Trust layers.

Three-layer architecture

1. Agent layer

AI agents built on the DKG Node, workflows orchestrated by Umanitek Guardian, or custom frameworks, communicating via MCP (Model Context Protocol) or A2A (Agent-to-Agent). Agents act as knowledge creators, verifiers, and brokers, reasoning over decentralized data.

2. Knowledge layer

The knowledge layer – OriginTrail Decentralized Knowledge Graph on NeuroWeb powered by Polkadot - is the connective tissue of the ecosystem.

3. Trust layer

Implements the economic and reputational backbone for trust. Projects may include tokenomics with TRAC, NEURO, DOT, or Polkadot-native tokens for staking, rewards, and reputation and/or x402 protocol for verifiable, machine-to-machine micropayments and gated data access

Submission requirements

1. GitHub repository

- a. Functional prototype, forked from the DKG Node repository
- b. Clear README with setup, dependencies, and architecture overview

- c. Documentation demonstrating use of DKG, MCP, x402, and NeuroWeb/Polkadot
- d. Structured examples (JSON-LD) and optional tokenomics simulation
- e. Clear demonstration of agent behavior (querying, verifying, or publishing to the DKG)

2. Video presentation (≤ 5 minutes)

- a. Problem statement and motivation
- b. Architecture overview of the Agent–Knowledge–Trust layer integration and use of the DKG Edge Node
- c. Live or recorded demo (agent behavior, DKG interaction, cross-chain data flow)
- d. Explanation of impact, ethical design, and scalability

Each team can only submit **one project**. Choose the challenge that best aligns with your aspirations, knowledge, and ideas to deliver your best work.

Judging framework

Core design principles

All submissions must demonstrate interoperability and value creation across the three foundational layers of the decentralized AI ecosystem:

- Agent layer
- Knowledge layer
- Trust layer

Projects should showcase how AI agents **leverage the OriginTrail DKG and the NeuroWeb** parachain on Polkadot to create **verifiable, interoperable, and autonomous multi-chain intelligence**. Projects might include agents that:

- Tap into cross-chain data sources
- Enable verifiable on-chain Knowledge Assets for Polkadot parachains

- Demonstrate how AI agents build on Polkadot's interoperability and shared security to empower the next generation of autonomous agents, from multi-chain DeFi advisors to AI-driven governance assistants and verifiable data oracles

Judging criteria

1. Excellence & innovation

Originality, conceptual rigor, and creative integration across all three layers.

Weight: 20%

Judges assess:

- Innovative multi-layer architecture linking agents, DKG, and trust mechanisms
- Originality of agent behavior: e.g., AI agents that detect misinformation, verify truth, or transact autonomously
- Use of Polkadot interoperability and shared security to empower new AI functions (cross-chain reasoning, governance, oracles)
- Depth of analysis and accuracy: sound judgment, factual precision, and clear reasoning behind claims or verifications
- Conceptual clarity and visual logic (architecture diagrams, reasoning workflows)
- Potential to advance the NeuroWeb-Polkadot trust layer

Scoring guide: 5 – Groundbreaking & technically advanced | 4 – Highly innovative & clear | 3 – Good but incremental | 2 – Limited originality | 1 – Weak concept

2. Technical implementation & code quality

Execution depth, interoperability, and clarity.

Weight: 40%

Judges assess:

- Functional implementation of your project
- Use of the DKG Edge Node (hard requirement)

- Use of DKG Knowledge Assets (discoverable, linked Knowledge Assets, valid JSON-LD/RDF)
- Code quality, documentation, and architecture clarity

Scoring guide: 5 – Fully functional & interoperable prototype | 4 – Working with minor issues | 3 – Prototype or partial integration | 2 – Limited functionality | 1 – Minimal execution

3. Impact & relevance

Societal, ecosystem, and multi-chain impact.

Weight: 20%

Judges assess:

- Contribution to the theme (Scaling Trust in the Age of AI) through cross-chain knowledge integrity
- Real-world value: misinformation defense, decentralized governance, identity verification, or transparent data economy
- Measurable outcomes – e.g., improved accuracy, verifiable provenance, or enhanced ecosystem usability

Scoring guide: 5 – Transformative multi-chain impact | 4 – Strong ecosystem contribution | 3 – Moderate but promising | 2 – Limited relevance | 1 – Minimal impact

4. Ethics, sustainability & openness

Responsible, transparent, and open collaboration.

Weight: 10%

Judges assess:

- Transparency and provenance (verifiable data, clear authorship, and auditability)
- Use of open-source standards and interoperability principles

- Alignment with human-centric values: openness, inclusion, and trustworthiness

Scoring guide: 5 – Exemplary ethical & sustainable design | 4 – Strong practices | 3 – Adequate awareness | 2 – Limited evidence | 1 – Absent consideration

5. Communication & presentation

Clarity and persuasiveness of storytelling.

Weight: 10%

Judges assess:

- Clarity in demonstrating three-layer interoperability
- Thoroughness of explanation – does the team clearly show how and why their system works?
- Highlight of key insights or discrepancies (where relevant)
- Visual clarity: effective use of diagrams, demos, and examples
- Narrative engagement: connection between technical execution and societal purpose

Scoring guide: 5 – Outstanding clarity & storytelling | 4 – Strong & engaging | 3 – Clear but unpolished | 2 – Hard to follow | 1 – Poor or incomplete

Terms & Conditions

Full Hackathon Terms & Conditions and Code of Conduct are available [here](#) ↗